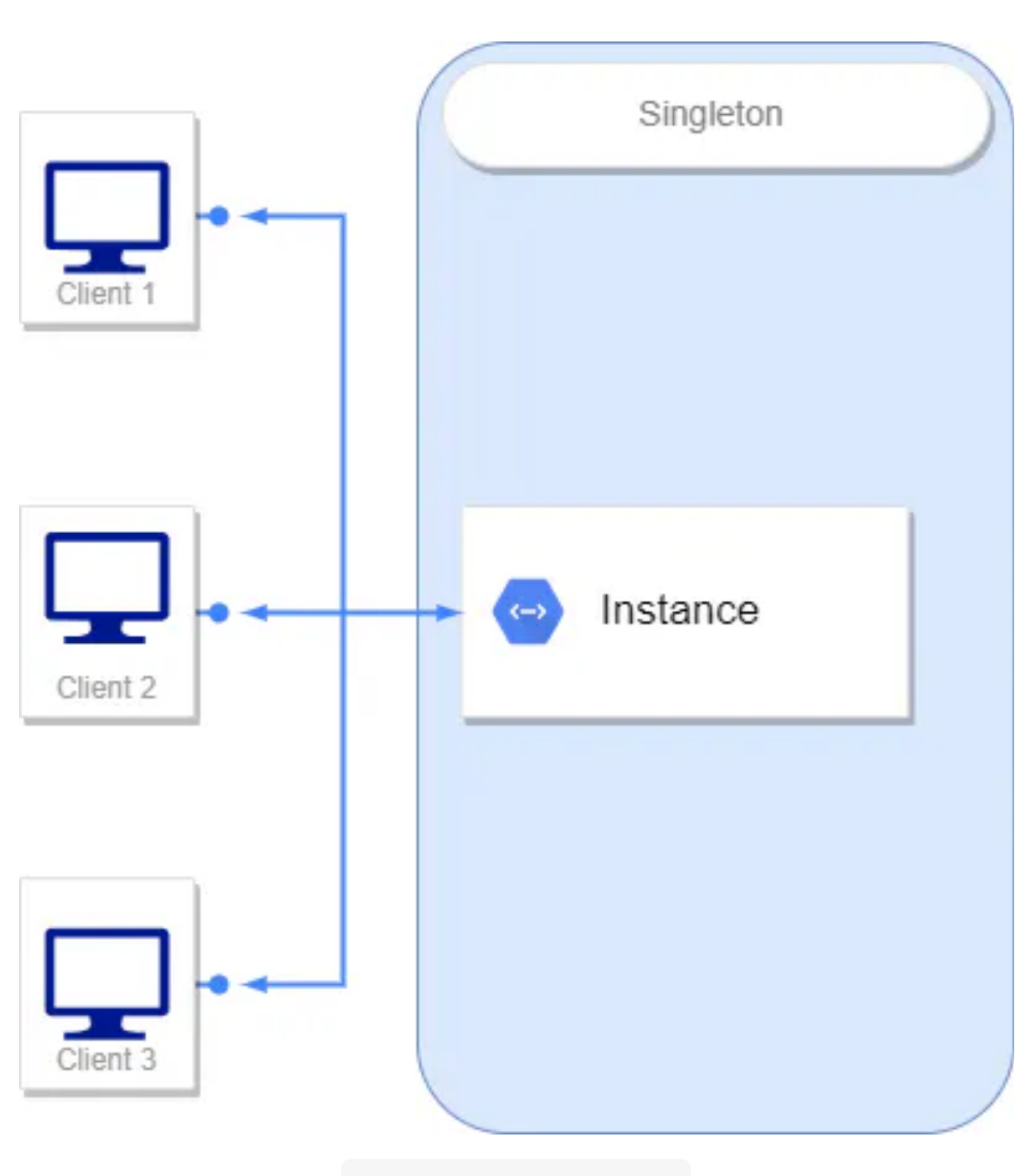
Singleton Pattern

The Singleton Pattern is a design pattern that restricts the instantiation of a class to one single instance. This is useful when exactly one object is needed to coordinate actions across the system. The Singleton pattern is recognized by a method that creates a new instance of a class if one doesn't exist. If an instance already exists, it simply returns a reference to that object.

Suppose multiple clients or parts of a system require access to a shared resource or service. There's a risk of creating multiple instances of the resource or service. This can lead to inefficiencies, inconsistent states, or conflict issues if each client creates its own instance. For example, if each client created its own database connection, it could overwhelm the database server and cause connection management chaos.



Singleton Pattern

The Singleton pattern addresses this problem by ensuring there is only one instance of a particular class (the Singleton), regardless of how many clients are trying to access it. In the above image, `Singleton` represents the class that enforces the single-instance rule, and `Instance` represents the single, shared instance. Each client (`Client 1`, `Client 2`, `Client 3`) is shown connecting to the same instance of the Singleton class, illustrating that they all share the same

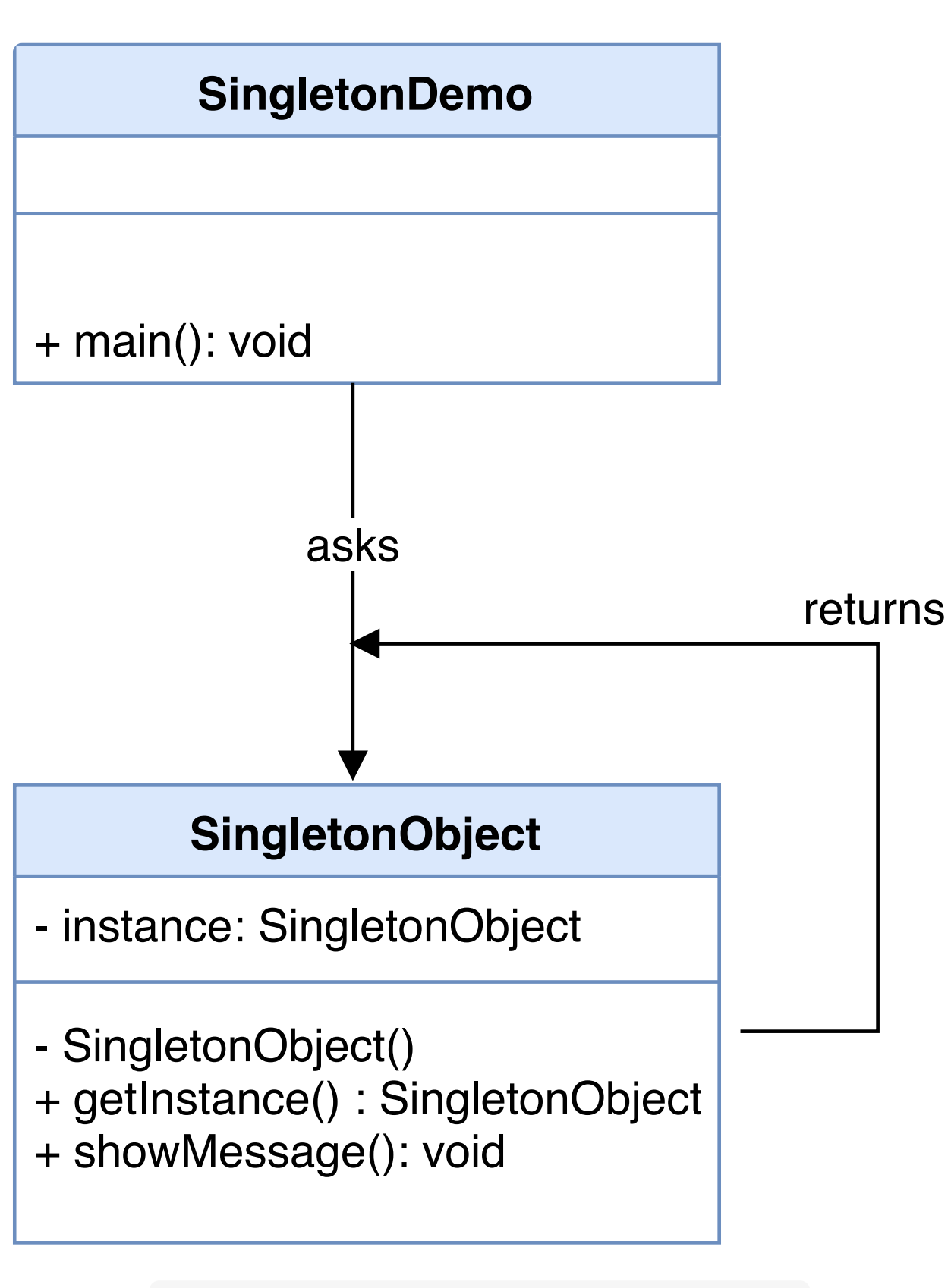
Real-World Example

In a practical scenario, Singleton pattern can be seen in a country's central government concept. A country has one central government. This government is responsible for imposing all the rules and regulations, administering the nation, and implementing the policies throughout the country. No matter how many institutions or individuals interact with the government, they all engage in the same single instance.

Singleton Pattern - Real-World Example

Structure of Singleton Pattern

Following is the class diagram of the Singleton Pattern.



Singleton Pattern - Class Diagram

1. **SingletonDemo Class**
    - This class contains the `main()` method, which serves as the entry point of the program. It's where the `SingletonObject` is accessed.
    - The `main()` method has a relationship with the `SingletonObject` class, indicated by the label "asks" on the arrow pointing from `SingletonDemo` to `SingletonObject`. This shows that `SingletonDemo` is requesting the Singleton instance.
2. **SingletonObject Class**
    - `instance`: A private static attribute of the `SingletonObject` type. This attribute holds the single instance of the `SingletonObject` class that will be shared across the system.
    - `SingletonObject()`: The constructor is private, which prevents instantiation from outside the class.
    - `getInstance()`: A public static method that returns the single instance of the `SingletonObject`. If the instance does not exist yet, this method will create it. This method implements the logic to ensure only one instance of `SingletonObject` is created.
    - `showMessage()`: A public instance method that `SingletonObject` can perform, typically to demonstrate that the instance is working.

The diagram also includes a return arrow labeled "returns" from `SingletonObject` to `SingletonDemo`, indicating that the instance of `SingletonObject` is returned to the `SingletonDemo` class after it is requested.

Implementation of Singleton Pattern

This section will discuss the implementation of the Singleton Pattern in four different languages: C++, Java, Python, and JavaScript.

In many programming languages, the Singleton pattern can be implemented by:

- Making the constructor private to prevent other objects from using the `new` operator with the Singleton class.
- Creating a static method that acts as a constructor. This method calls the private constructor to create an object and saves it in a static field. All following calls to this method return the cached object.

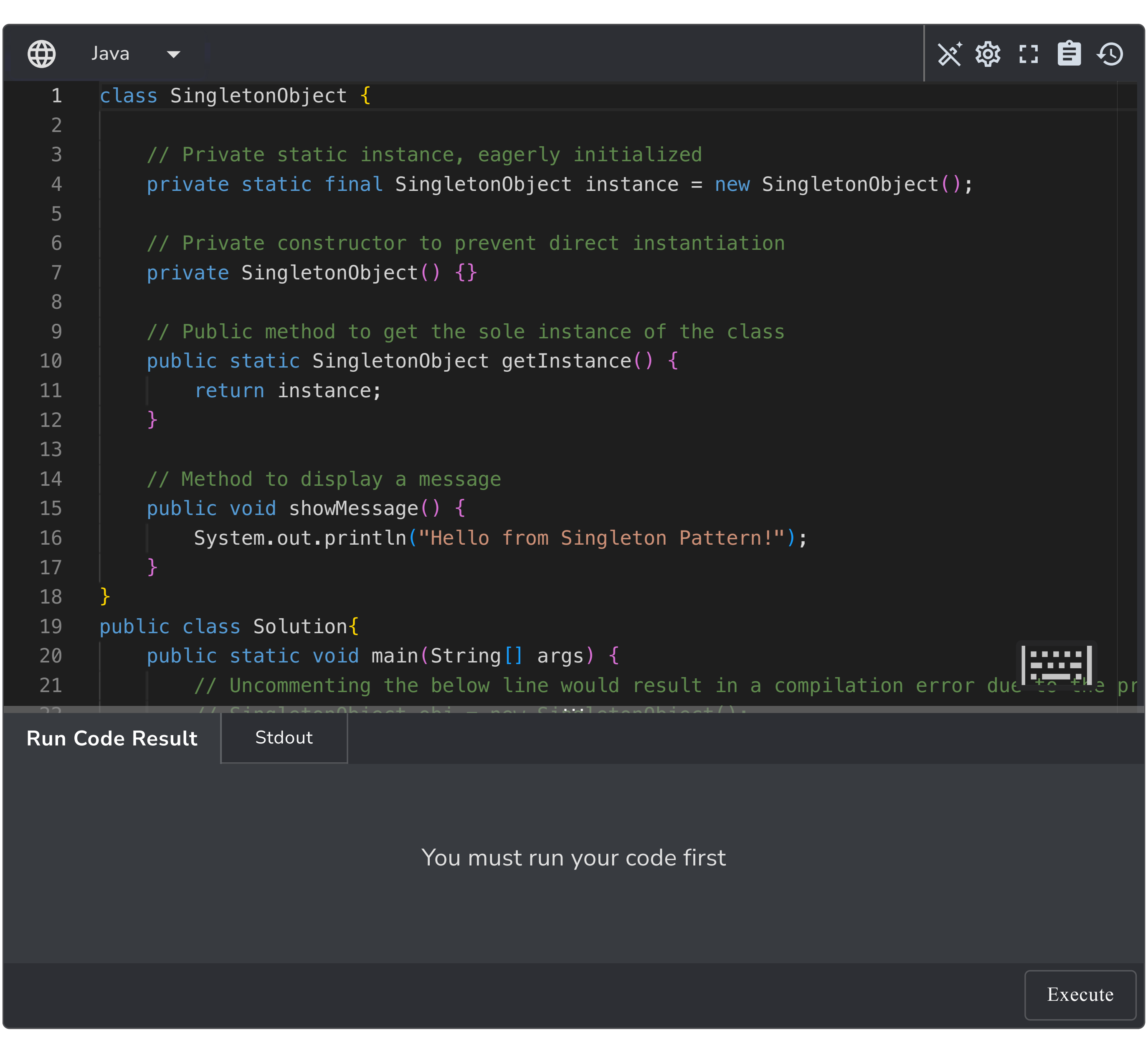
Pseudocode

Let's first discuss the pseudocode for Singleton Pattern, then we will discuss its implementation in different languages.

We will create two classes, `SingletonObject` and `SingletonDemo`. `SingletonObject` is the class that implements the Singleton Pattern, and the demo class will use this class to create an object.

```
Class SingletonObject:

    // Declare a private static instance and initialize it
    Create a private, static, final instance of SingletonObject

    // Private constructor
    Constructor:
        It is private, so it cannot be called from outside the class

    // Method to return the unique instance of this class
    Function getInstance() → SingletonObject:
        Return the unique instance

    // Method to display a message
    Function showMessage():
        Print "Hello from Singleton Pattern!"

End Class

Main Program:

    // Get the unique instance of SingletonObject
    Declare object = SingletonObject.getInstance()

    // Call the showMessage method
    object.showMessage()

End Main Program
```

- **Private Constructor**: A private constructor is available in the `SingletonObject` class. This ensures that no instance of this class can be made from a source outside the class.
- **Static Instance**: A `SingletonObject` has a private, already initialized static final instance. This indicates that the instance is created when the class loads. This instance cannot be changed because it is final.
- `getInstance()` **Function**: The `getInstance` function gives users global access to that particular instance. The singleton object's singular instance can be retrieved through this global access point.
- `showMessage()` **Function**: The simple `showMessage` method prints a message to the console.



```java
class SingletonObject {

    // Private static instance, eagerly initialized
    private static final SingletonObject instance = new SingletonObject();

    // Private constructor to prevent direct instantiation
    private SingletonObject() {}

    // Public method to get the sole instance of the class
    public static SingletonObject getInstance() {
        return instance;
    }

    // Method to display a message
    public void showMessage() {
        System.out.println("Hello from Singleton Pattern!");
    }
}
public class Solution{
    public static void main(String[] args) {
        // Uncommenting the below line would result in a compilation error due to the pr
```

You must run your code first

Applications of Singleton Pattern

In software design, the Singleton Pattern is an age-old favorite pattern to use in many scenarios. Let's discuss some of the common scenarios where it is used:

Configuration Management:

Almost all of the software applications need some configuration settings to run. These settings should remain consistent throughout the application scope, whether they are loaded from a file or server. Using a Singleton Pattern for configuration management allows us to make sure that:

- There is a single source for configuration management.
- Settings are initialized and loaded only once throughout the application lifecycle.
- Various parts of the application can access these settings consistently.

Database Connection Pools:

Efficient management of database connections is essential for the performance of an application. Singleton can help us with the following:

- Rather than constantly opening and closing new database connections, it can reuse a pool of them.
- It ensures efficient utilization and management with limited database connections.
- It offers a consistent mechanism to access the database through various application parts.

Hardware Access Management:

Singleton can help us with systems interfacing with some hardware resources like printers or graphics cards with the following:

- Ensures there is synchronized access to the hardware, which prevents potential conflicts.
- Offers a consistent interface for managing and controlling the hardware resource.
- It manages the initialization and shutdown routine for the hardware, which reduces the overhead.

These are examples of scenarios where Singleton Pattern can help us achieve efficiency in our software application.

Pros and Cons

Every design pattern offers numerous advantages, but it has some drawbacks as well. Let's look at the pros and cons of Singleton Pattern:

Pros	Cons
Controlled Instance Access: Ensures that only one instance of a class is created and provides a single point of access to it.	**Global State:** Encourages the use of a global state in an application, which can be risky and hard to debug.
Lazy Initialization: The instance is only created when it is needed, potentially saving resources if the instance is never used.	**Testing Challenges:** Can introduce difficulties in testing because it's hard to mock a singleton and it carries state across tests.
Subclassing Flexibility: Can be subclassed, and the application can configure the subclass to use at runtime.	**Tight Coupling:** The pattern often gets tightly coupled with the rest of the application code, which can be problematic for maintenance.
Instance Control: Can be extended to control the number of instances that the application uses.	**Scalability Issues:** Can become a bottleneck for scaling applications, particularly in multi-threaded application scenarios.
Reduced Namespace Pollution: Avoids polluting the global namespace with global variables that hold instances.	**Refactoring Hurdles:** Refactoring a class from a singleton to a non-singleton can require significant changes if the global access point has been widely used.
Shared Resources Management: Useful for managing shared resources, such as a connection to a database.	**Hidden Dependencies:** Introduces hidden dependencies within the application's classes, which can lead to unexpected behaviors and issues.

Summary

In software engineering, the singleton pattern is a popular design pattern that ensures a class has only one instance and offers a global point of access to this instance. It has benefits like lazy initialization, fewer global namespace issues, and restricted access to a single resource. However, it has certain disadvantages, including potential for tight coupling, challenges with testing, scalability, and the introduction of hidden dependencies within an application. It should be used carefully, taking into account the influence on the design and maintainability of the program, even though it can be a useful pattern for managing shared resources and configurations.

← Previous
Introduction

Next →
Factory Method Pattern

☐ Mark as Completed